

Go Testing & Verification Handbook

Reviewer Guide: Unit Testing, Fixtures, Doubles, Integration & Fuzzing

1. Unit Testing Mechanics

Table-Driven Anatomy: Structure test cases using anonymous struct slices with explicit give and want fields. Table-driven tests organize inputs, edge cases, and expected outputs cleanly in a single readable execution structure.

Subtests (t.Run): Execute each table row inside an isolated `t.Run(tt.name, func(t *testing.T) { ... })` block. This delivers distinct failure reports for individual test cases and allows developers to run specific subtests via the CLI (`go test -run TestX/SubCase`).

Loop Variable Capture (Go 1.22+): In Go 1.22+, loop variable scope gotchas in `for` range loops are resolved natively by the compiler. Redundant `tt := tt` re-declarations prior to invoking `t.Parallel()` inside subtests are obsolete.

Assertions & Diffs: Always enforce the got before want checking order (if `got != tt.want`). For complex structs, slices, or maps, use `google/go-cmp (cmp.Diff)` rather than fragile `reflect.DeepEqual` to get clear, line-by-line diff reports upon failure.

Error Attribution: Use `t.Error` / `t.Errorf` for individual table row failures to allow remaining table cases to run. Reserve `t.Fatal` / `t.Fatalf` strictly for unrecoverable setup failures (e.g., failed fixture initialization) that render subsequent tests invalid.

Idiomatic Table-Driven Subtest Example:

```
func TestCalculateDiscount(t *testing.T) {
    tests := []struct {
        name      string
        give       Order
        want       Amount
        wantErr    error
    }{
        {name: "valid order", give: Order{Total: 100}, want: Amount(10), wantErr: nil},
        {name: "invalid total", give: Order{Total: -5}, want: 0, wantErr: ErrInvalidTotal},
    }
    for _, tt := range tests {
        t.Run(tt.name, func(t *testing.T) {
            t.Parallel() // Go 1.22+: No 'tt := tt' needed
            got, err := CalculateDiscount(tt.give)
            if !errors.Is(err, tt.wantErr) {
                t.Fatalf("err = %v, want %v", err, tt.wantErr)
            }
            if diff := cmp.Diff(tt.want, got); diff != "" {
                t.Errorf("CalculateDiscount() mismatch (-want +got):\n%s", diff)
            }
        })
    }
}
```

2. Helpers, Fixtures & Lifecycle Management

Call-Site Attribution: Annotate test helper routines with `t.Helper()` at the top of the function body. This instructs the testing framework to omit the helper frame from failure stack traces, attributing line numbers directly to the caller's test file line.

Teardown Hooks (`t.Cleanup`): Register resource teardown tasks directly using `t.Cleanup(func() { ... })` inside helpers and test setup routines. `t.Cleanup` executes reliably in LIFO order upon test completion, preventing resource leaks even if a test calls `t.Fatal()`.

Environment & Temp Directories: Use `t.TempDir()` to obtain parallel-safe, self-cleaning temporary directory paths. Mutate environment variables strictly using `t.Setenv(key, val)`, which automatically restores the original environment value when the test finishes.

Context Lifecycle (Go 1.24+): In Go 1.24+, bind test context cancellation directly to test teardown using `t.Context()`. This eliminates manual `context.WithCancel` boilerplate and ensures all derived background operations cancel cleanly upon test termination.

Idiomatic Helper & Lifecycle Pattern:

```
func setupTestDB(t *testing.T) *sql.DB {
    t.Helper() // Attributes errors to caller's line
    db, err := sql.Open("sqlite3", t.TempDir()+"/test.db")
    if err != nil {
        t.Fatalf("failed to open test db: %v", err)
    }
    t.Cleanup(func() { _ = db.Close() }) // Reliable teardown
    return db
}
```

3. Test Doubles & Boundaries

Inline Hand-Crafted Fakes: Prefer lightweight, hand-crafted stub or fake structs written directly inside `_test.go` files over generating heavy, auto-generated mock frameworks (e.g., gomock). Hand-written fakes keep tests readable, fast, and resilient to refactoring.

Function-Based Injection: Inject dynamic behavior or time dependencies using higher-order functions (e.g., passing now `func() time.Time` as a struct field or constructor parameter). This enables precise time mocking without requiring external clock abstractions.

Black-Box Boundaries (`package foo_test`): Standardize on `package foo_test` for unit test files to enforce black-box verification of public API contracts. Restrict `package foo` test declarations strictly to testing unexported helper functions or private state transitions.

Function Injection & Inline Stub Pattern:

```
type TokenGenerator struct {
    Now func() time.Time // Injected time source
}

func TestTokenExpiry(t *testing.T) {
    fixedTime := time.Date(2026, 1, 1, 12, 0, 0, 0, time.UTC)
    gen := &TokenGenerator{Now: func() time.Time { return fixedTime }}
    tok := gen.Generate()
    if tok.CreatedAt != fixedTime {
        t.Errorf("got %v, want %v", tok.CreatedAt, fixedTime)
    }
}
```

4. Integration & Database Testing

Build Tags & Short Skipping: Segregate slow-running integration test suites using the `//go:build` integration directive at the top of test files, or skip them during quick local runs using `if testing.Short() { t.Skip("skipping integration test") }`.

Database Isolation & Transactions: Run integration tests against real ephemeral database instances managed via Testcontainers. To ensure strict isolation between concurrent tests, execute each test within an isolated database transaction that rolls back automatically inside `t.Cleanup()`.

Network Isolation (`httptest.Server`): Mock external HTTP dependencies locally using the standard library's `httptest.NewServer` or `httptest.NewTLSServer`. Never make real outbound network requests during automated unit or integration test runs.

Transactional Database Test Pattern:

```
func TestCreateUser_Integration(t *testing.T) {
    if testing.Short() { t.Skip("skipping integration test") }
    tx, err := testDB.BeginTx(t.Context(), nil)
    if err != nil { t.Fatalf("begin tx failed: %v", err) }
    t.Cleanup(func() { _ = tx.Rollback() }) // Auto rollback

    repo := NewUserRepository(tx)
    if err := repo.Create(t.Context(), &User{Name: "Alice"}); err != nil {
        t.Fatalf("create user failed: %v", err)
    }
}
```

5. Concurrency, Benchmarks & Flakiness Prevention

Race Detection (-race): Mandate `go test -race ./...` across all CI/CD pipelines and local test executions. The Go race detector instrumented runtime detects non-deterministic memory accesses and unsynchronized concurrent map mutations before production deployment.

Allocation Benchmarking (`testing.B`): Write performance benchmarks using `func BenchmarkX(b *testing.B)`. Call `b.ResetTimer()` after costly setup routines and invoke `b.ReportAllocs()` to track heap allocation counts and byte volumes across iteration loops.

Timing Controls & Flakiness: Prohibit non-deterministic `time.Sleep()` calls in concurrent test assertions. Synchronize asynchronous operations strictly using channels, `sync.WaitGroup`, or bounded polling loops (e.g., checking condition every 10ms with timeout).

6. Security & Native Fuzz Testing

Dependency Vulnerability Scanning: Integrate `govulncheck ./...` into continuous integration pipelines to identify known CVE vulnerabilities in third-party module dependencies and standard library calls.

Native Fuzz Testing (`testing.F`): Write native Go fuzz tests (`func FuzzParseJSON(f *testing.F)`) to discover parser crashes, buffer overflows, and unhandled panic boundaries on random inputs generated by the coverage-guided fuzz engine.

Native Go Fuzzing Pattern:

```
func FuzzParseConfig(f *testing.F) {
    f.Add([]byte("key=value")) // Seed corpus
    f.Fuzz(func(t *testing.T, data []byte) {
```

```

    _, _ = ParseConfig(data) // Must not panic
  })
}

```

7. Code Reviewer Quick-Reference Grading Matrix

Use this checklist to grade Go testing suites during pull request code reviews:

Review Check	Anti-Pattern / Code Smell	Idiomatic Remediation
Table-Driven Tests	Multiple repetitive TestX functions testing same logic with different inputs.	Consolidate cases into a table slice with give/want struct fields.
Loop Captures	Redundant 'tt := tt' bindings inside for range loops in Go 1.22+ codebase.	Remove obsolete variable capture line; loop scope is natively safe.
Helper Attribution	Test failures pointing to internal helper line instead of caller test line.	Add t.Helper() at the top of setup/utility functions.
Teardown Safety	Using defer inside helper functions or manual cleanup code at end of test.	Register cleanup callbacks using t.Cleanup() immediately after allocation.
Env & File Isolation	os.Setenv() or os.MkdirTemp() without manual cleanup or restoration.	Use t.Setenv() and t.TempDir() for automatic, thread-safe cleanup.
Mocking Overhead	Auto-generating thousands of lines of complex mock frameworks for 1 method.	Write a 5-line hand-crafted fake or inject a higher-order function.
Public API Scope	Declaring all tests in 'package foo', exposing internal state to test cases.	Use 'package foo_test' for black-box testing public API contracts.
Database Isolation	Tests mutating shared DB state without cleanup, causing cross-test failures.	Wrap test in DB transaction and register tx.Rollback() inside t.Cleanup().
Flaky Timing	Using time.Sleep(100*time.Millisecond) to wait for goroutine completion.	Synchronize using channels, WaitGroup, or bounded polling loops.
Fuzzing Coverage	Relying solely on static unit test inputs for complex string/binary parsers.	Add native Go fuzz test (testing.F) with seed corpus.